

第 8 章：反向传播

第 1 节：梯度计算 (Evaluation of Gradients)

《深度学习基础》课程小组

2026 年 10 月 7 日

目录

- 1 引言
- 2 单层网络
- 3 一般前馈网络
- 4 简单示例
- 5 数值微分法
- 6 雅可比矩阵
- 7 黑塞矩阵
- 8 总结
- 9 参考文献

1. 反向传播：目标与动机

- 目标：为前馈神经网络寻找**高效计算误差函数梯度 $E(\mathbf{w})$** 的技术。
- 方法：**误差反向传播** (error backpropagation)，信息从后向前传递。
- 历史上：反向传播方程手工推导并手写实现，耗时且易错。
- 现代：**自动微分** (automatic differentiation) 几乎可以计算任何导数。
- 但仍然值得理解计算过程，避免依赖“黑箱”软件。

术语说明

“反向传播”在文献中有多种用法。本书特指**数值计算导数的计算过程**，而非端到端训练过程。

1.1 误差函数的求和形式

- 许多实际误差函数（如 i.i.d. 数据的最大似然）是各项之和：

$$E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w}) \quad (8.1)$$

- 我们考虑评估其中一项 $\nabla E_n(\mathbf{w})$ 。
- 可直接用于 SGD，或对批量/小批量方法累加。

2. 单层网络 (8.1.1)

考虑简单线性模型：

$$y_k = \sum_i w_{ki} x_i \quad (8.2)$$

平方和误差：

$$E_n = \frac{1}{2} \sum_k (y_{nk} - t_{nk})^2 \quad (8.3)$$

梯度：

$$\frac{\partial E_n}{\partial w_{ji}} = (y_{nj} - t_{nj}) x_{ni} \quad (8.4)$$

- 可解释为 **“局部” 计算**：误差信号 $(y_{nj} - t_{nj})$ 与输入 x_{ni} 的乘积。
- 这一简单结果将推广到多层前馈网络。

3. 一般前馈网络 (8.1.2)

- 每个单元计算输入的**加权和**:

$$a_j = \sum_i w_{ji} z_i \quad (8.5)$$

- 通过**非线性激活函数** $h(\cdot)$ 得到激活值:

$$z_j = h(a_j) \quad (8.6)$$

- 偏置可通过引入固定 +1 的额外单元包含在求和中。
- 前向传播**: 给定输入, 依次应用 (8.5)(8.6) 计算所有隐藏和输出单元。

3.1 误差项 δ_j 的定义

考虑 E_n 关于权重 w_{ji} 的导数。 E_n 仅通过 a_j 依赖于 w_{ji} :

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial w_{ji}} \quad (8.7)$$

引入**误差项**:

$$\delta_j \equiv \frac{\partial E_n}{\partial a_j} \quad (8.8)$$

由 (8.5) 得:

$$\frac{\partial a_j}{\partial w_{ji}} = z_i \quad (8.9)$$

代入得:

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i \quad (8.10)$$

- 导数 = **输出端 δ** $\times$ **输入端 z** (与线性模型 (8.4) 同形式)。

3.2 输出单元与隐藏单元的 δ

输出单元 (使用规范链接激活函数):

$$\delta_k = y_k - t_k \quad (8.11)$$

隐藏单元: 再次使用链式法则:

$$\delta_j \equiv \frac{\partial E_n}{\partial a_j} = \sum_k \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j} \quad (8.12)$$

代入定义并利用 (8.5)(8.6):

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k \quad (8.13)$$

- 隐藏单元的 δ 通过反向传播输出单元的 δ 得到。
- 求和遍历 w_{kj} 的第一个下标 (反向), 而前向传播遍历第二个下标。

3.3 反向传播算法伪代码

Algorithm 1 Backpropagation

Input: 输入向量 $\mathbf{x}_n$; 网络参数 $\mathbf{w}$; 误差函数 $E_n(\mathbf{w})$; 激活函数 h

Output: 误差函数导数 $\{\partial E_n / \partial w_{ji}\}$

// 前向传播

for $j \in$ 所有隐藏和输出单元 do

$$a_j \leftarrow \sum_i w_{ji} z_i$$

// 包含输入

$$z_j \leftarrow h(a_j)$$

end for

// 误差评估

for $k \in$ 所有输出单元 do

$$\delta_k \leftarrow \frac{\partial E_n}{\partial a_k}$$

end for

// 反向传播 (逆序)

for $j \in$ 所有隐藏单元 do

$$\delta_j \leftarrow h'(a_j) \sum_k w_{kj} \delta_k$$

$$\frac{\partial E_n}{\partial w_{ji}} \leftarrow \delta_j z_i$$

end for

return $\{\frac{\partial E_n}{\partial w_{ji}}\}$

- 批量方法: 对每个数据点重复, 然后求和: $\frac{\partial E}{\partial w_{ji}} = \sum_n \frac{\partial E_n}{\partial w_{ji}}$

4. 简单示例 (8.1.3)

两层网络 + 平方和误差：

- 输出单元：线性激活， $y_k = a_k$
- 隐藏单元：sigmoidal 激活， $h(a) = \tanh(a)$
- 导数： $h'(a) = 1 - h(a)^2$

前向传播

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i, \quad z_j = \tanh(a_j), \quad y_k = \sum_{j=0}^M w_{kj}^{(2)} z_j.$$

其中 $x_0 = z_0 = 1$ 用于包含偏置。

4.1 反向传播与导数

输出单元误差：

$$\delta_k = y_k - t_k \quad (8.21)$$

隐藏单元误差：

$$\delta_j = (1 - z_j^2) \sum_{k=1}^K w_{kj}^{(2)} \delta_k \quad (8.22)$$

权重导数：

$$\frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j x_i, \quad \frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k z_j \quad (8.23)$$

- δ_k ：输出层误差； δ_j ：隐藏层误差。

5. 数值微分法 (8.1.4)

- 反向传播的重要特性：**计算效率**。
- 单次误差函数评估需 $\mathcal{O}(W)$ 次操作。
- 前向传播中大部分计算来自 (8.5) 的求和，激活函数评估开销小。
- 求和每项需一次乘法和一次加法，总成本 $\mathcal{O}(W)$ 。

有限差分法

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n(w_{ji} + \epsilon) - E_n(w_{ji})}{\epsilon} + \mathcal{O}(\epsilon) \quad (8.24)$$

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n(w_{ji} + \epsilon) - E_n(w_{ji} - \epsilon)}{2\epsilon} + \mathcal{O}(\epsilon^2) \quad (8.25)$$

- 中心差分法： $\mathcal{O}(\epsilon)$ 修正项抵消，精度更高。
- 但计算步数约为 (8.24) 的**两倍**。

5.1 数值微分的问题与用途

- **主要问题**：失去 $\mathcal{O}(W)$ 缩放特性。
 - 每次前向传播 $\mathcal{O}(W)$ 步。
 - 需扰动 W 个权重，总成本 $\mathcal{O}(W^2)$ 。
- **实际用途**：检验软件正确性。
 - 将反向传播或自动微分的导数与**中心差分**结果比较。
 - 是验证实现正确性的有力手段。

6. 雅可比矩阵 (8.1.5)

- 反向传播也可计算其他导数，如雅可比矩阵：

$$J_{ki} \equiv \frac{\partial y_k}{\partial x_i} \quad (8.26)$$

- 每个导数在其他输入固定时计算。
- 雅可比矩阵在模块化系统中作用重要 (图 8.3)。
- 每个模块可以是固定或可学习函数，线性或非线性，只要可微。

雅可比矩阵的用途

- 衡量输出对输入变化的局部敏感度
- 支持模块化系统中的误差传播
- 用于近似输出变化与不确定性传播
- 作为高阶导数计算的基础

6.1 雅可比矩阵的反向传播计算

- 误差函数导数:

$$\frac{\partial E}{\partial w} = \sum_{k,j} \frac{\partial E}{\partial y_k} \frac{\partial y_k}{\partial z_j} \frac{\partial z_j}{\partial w} \quad (8.27)$$

中间项即为红色模块的雅可比矩阵。

- 输入误差传播:

$$\Delta y_k \approx \sum_i \frac{\partial y_k}{\partial x_i} \Delta x_i \quad (8.28)$$

仅在小扰动时有效，需对每个新输入重新计算。

6.2 雅可比矩阵的递归公式

将 J_{ki} 写成:

$$J_{ki} \equiv \frac{\partial y_k}{\partial x_i} = \sum_j \frac{\partial y_k}{\partial a_j} \frac{\partial a_j}{\partial x_i} = \sum_j w_{ji} \frac{\partial y_k}{\partial a_j} \quad (8.29)$$

递归反向传播公式:

$$\frac{\partial y_k}{\partial a_j} = \sum_{\ell} \frac{\partial y_k}{\partial a_{\ell}} \frac{\partial a_{\ell}}{\partial z_j} = h'(a_j) \sum_{\ell} w_{\ell j} \frac{\partial y_k}{\partial a_{\ell}} \quad (8.30)$$

- 从输出单元开始, 所需导数可直接从输出激活函数得到。
- **线性输出**: $\partial y_k / \partial a_{\ell} = \delta_{k\ell}$
- **Sigmoid 输出**: $\partial y_k / \partial a_{\ell} = \delta_{k\ell} \sigma'(a_k)$
- **Softmax 输出**: $\partial y_k / \partial a_{\ell} = \delta_{k\ell} y_{\ell} - y_k y_{\ell}$

6.3 雅可比矩阵的计算流程与检验

计算流程：

- ① 前向传播获取所有隐藏和输出单元状态。
- ② 对雅可比矩阵每一行 k ，用 (8.30) 反向传播，起点为 (8.31)(8.33)(8.34)。
- ③ 用 (8.29) 传播至输入层。

前向传播形式：也可用类似方式推导，适合输入维度小、输出维度大的情况。数值检验：

$$\frac{\partial y_k}{\partial x_i} = \frac{y_k(x_i + \epsilon) - y_k(x_i - \epsilon)}{2\epsilon} + \mathcal{O}(\epsilon^2) \quad (8.35)$$

- 需 $2D$ 次前向传播，总计算量 $\mathcal{O}(DW)$ 。
- 效率远低于反向传播，但可用于验证实现正确性。

7. 黑塞矩阵 (8.1.6)

- 反向传播也可计算二阶导数：

$$\frac{\partial^2 E}{\partial w_{ij} \partial w_{kl}} \quad (8.36)$$

- 将所有权重和偏置视为向量 $\mathbf{w}$ 的元素，二阶导数构成黑塞矩阵：

$$H_{ij} = \frac{\partial^2 E}{\partial w_i \partial w_j} \quad (8.37)$$

- 应用于：二阶优化算法、贝叶斯方法、大语言模型权重精度降低。

7.1 黑塞矩阵的计算复杂度

- 黑塞矩阵维度 $W \times W$, 计算复杂度 $\mathcal{O}(W^2)$ (每个数据点)。
- 扩展反向传播可高效计算, 缩放为 $\mathcal{O}(W^2)$ 。
- 有时只需黑塞矩阵与向量的乘积 $\mathbf{v}^T \mathbf{H}$:
 - 可用反向传播扩展在 $\mathcal{O}(W)$ 步内计算。

实际困难

- 神经网络可能有数百万甚至数十亿参数。
- 完整黑塞矩阵的评估或存储不可行。
- 求逆更苛刻: $\mathcal{O}(W^3)$ 复杂度。
- 因此需要近似替代方法。

7.2 对角近似与外积近似

对角近似:

- 仅计算对角线元素, 非对角线设为零。
- 存储 $\mathcal{O}(W)$, 求逆 $\mathcal{O}(W)$, 但计算仍需 $\mathcal{O}(W^2)$ 。
- 实际中黑塞矩阵通常有显著非对角线项, 需谨慎使用。

外积近似 (Levenberg-Marquardt 近似):

- 对平方和误差: $\mathbf{H} = \sum_{n=1}^N \nabla y_n (\nabla y_n)^T + \sum_{n=1}^N (y_n - t_n) \nabla \nabla y_n$
- 当网络训练良好时, 第二项很小可忽略: $\mathbf{H} \simeq \sum_{n=1}^N \nabla a_n \nabla a_n^T$
- 仅涉及一阶导数, 可用标准反向传播 $\mathcal{O}(W)$ 计算, 矩阵元素 $\mathcal{O}(W^2)$ 。
- 交叉熵 + sigmoid 输出: $\mathbf{H} \simeq \sum_{n=1}^N y_n (1 - y_n) \nabla a_n \nabla a_n^T$

8. 本节总结

- **反向传播**: 高效计算误差函数梯度, 复杂度 $\mathcal{O}(W)$ 。
- **单层网络**: 导数 = 误差信号 $\times$ 输入, 形式简单。
- **一般前馈网络**:
 - 定义误差项 $\delta_j = \partial E_n / \partial a_j$
 - 输出单元: $\delta_k = y_k - t_k$
 - 隐藏单元: $\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$
 - 权重导数: $\partial E_n / \partial w_{ji} = \delta_j z_i$
- **数值微分**: 简单可靠但 $\mathcal{O}(W^2)$, 用于验证实现正确性。
- **雅可比矩阵**: 输出对输入的导数, 用于敏感度分析和模块化系统。
- **黑塞矩阵**: 二阶导数矩阵, 用于二阶优化和贝叶斯方法。
 - 完整计算 $\mathcal{O}(W^2)$, 求逆 $\mathcal{O}(W^3)$ 。
 - 外积近似仅需一阶导数, 是重要的实用近似。

9. 参考文献

- 教材: Bishop, C. M. & Bishop, H. (2023). Deep Learning Foundations and Concepts. Springer Nature Switzerland AG.
- Bishop, C. M. (1992). Exact calculation of the Hessian matrix within a feedforward network.
- Møller, M. F. (1993). A scaled conjugate gradient algorithm for fast supervised learning.
- Pearlmutter, B. A. (1994). Fast exact multiplication by the Hessian.
- MacKay, D. J. C. (1992). A practical Bayesian framework for backpropagation networks.
- Shen, S. et al. (2019). Q-BERT: Hessian based ultra low precision quantization of BERT.